

Doubly Linked List – Traverse In Order

–Space Complexity and Algorithm

Program :

```
void traverseInOrder()
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    Node *temp = head;
    cout << "List (Forward): NULL <- ";
    while (temp != nullptr)
    {
        cout << temp->data << " <-> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}
... .
case 10:
    traverseInOrder();
    break;
```

Space Complexity

Auxiliary Space

Auxiliary space is the extra or temporary memory used by a function, not including the input data itself.

For the traverseInOrder function, the only new variable created is:

Node *temp: A single temporary pointer is used to iterate through the list. The memory required for a pointer is constant.

The function does not allocate any new nodes or data structures. Since the amount of extra memory is fixed, the auxiliary space complexity is $O(1)$.

Auxiliary Space: $O(1)$

Input Space

Input space is the memory used to store the function's inputs. This can be viewed in two ways.

1. Parameter – only Definition

This perspective considers only the space taken by the function's direct parameters. The function signature is `void traverseInOrder()`.

The function takes no parameters.

Under this definition, the input space is constant.

Input Space (Parameter-only): $O(1)$

2. Full – Data Definition

This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list.

If the linked list contains n nodes, the total memory it occupies is proportional to n .

Under this definition, the input space is linear with respect to the size of the list.

Input Space (Full-Data): $O(n)$

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, the "space complexity" of a function refers to its auxiliary space. Therefore, the most common and expected answer for this function is $O(1)$. The while loop that travels the list affects the function's time complexity ($O(n)$), but not its space complexity.

Algorithm: Traverse and Display List (Forward)

DTRAVERSE(START)

1. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

Write: "List is empty."

EXIT.

[End of If structure.]

2. [INITIALIZE POINTER AND PRINT HEADER]

a. Set PTR := START [PTR is a pointer used to traverse the list].

b. Write: "List (Forward): NULL < – "

3. [TRAVERSE THE LIST AND PRINT EACH ELEMENT]

Repeat while PTR ≠ NULL:

a. Write: INFO[PTR], " < –> " [Print the data of the current node].

b. Set PTR := NEXTLINK[PTR] [Move to the next node].

[END of Loop]

4. [PRINT THE LIST TERMINATOR]

Write: "NULL"

5. Exit
